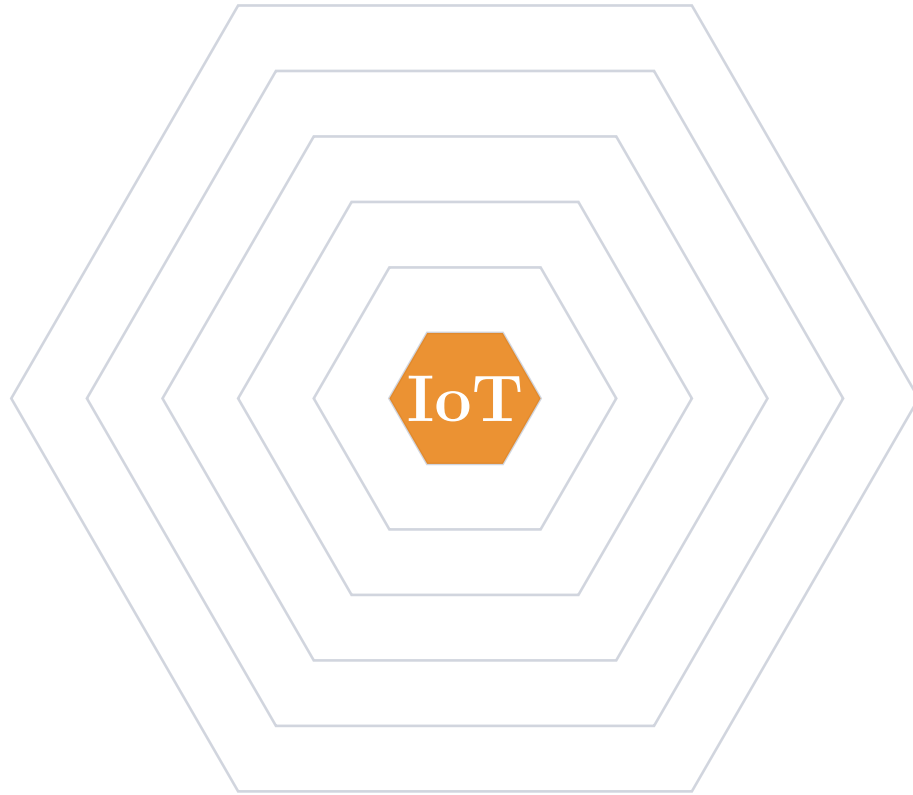




Aegis-IoT

AnyLogic Native Execution Runbook

A detailed, step-by-step beginner's guide to deploying the complete AI-powered network simulation pipeline natively.

Components Covered:

AnyLogic Native Simulation · Java Swing Dashboards · M2CGen Transpilation
Reinforcement Learning (Q-Tables) · Federated Learning Hub

Date: May 2026

Contents

1	Step 1: Check System Prerequisites	2
2	Step 2: Install Project Dependencies	3
3	Step 3: Synthesize ML Models into Java (M2CGen)	3
4	Step 4: Inject UI and RL Features into the AnyLogic Core	4
5	Step 5: Launch the Core AnyLogic Simulation	4
6	Step 6: Run the Federated Learning Scenario	5
7	Final Validation	6

Introduction to the Pipeline

This runbook provides complete, detailed instructions for compiling, deploying, and validating the Aegis-IoT environment completely within the AnyLogic engine. Our entire system operates directly out of the Java-based AnyLogic instance and Python logic—removing any external web dependencies.

Aegis-IoT relies on four main pillars: Data/ML preparation (Python), Code Injection (Python), the Core Simulation Engine (AnyLogic JVM), and Native Visualization Interfaces (Java Swing).

Step 1: Check System Prerequisites

To run this stack natively, you need local installations of specific tools. Ensure the following are available on your system path. Open a terminal (PowerShell, Command Prompt, or VS Code terminal) and verify:

Terminal / Command Prompt (Checking Versions)

```
python -version java -version
```

- **Python (3.8+):** Required for training the Machine Learning models (Scikit-Learn), running the XML mutation scripts, and executing the Federated Learning scenarios.
- **Java JDK (11+):** Required by the AnyLogic engine to compile and run the generated bytecode and custom Swing dashboards.
- **AnyLogic 8.x:** You must have the AnyLogic IDE installed to actually open, compile, and run the `.alpx` modeling file natively.

Step 2: Install Project Dependencies

We now need to install the external Python libraries required for our AI agents. Ensure your terminal is in the root directory (`FinalCCNProject`), then run:

```
Terminal / Command Prompt (Python Packages)
```

```
pip install -r requirements.txt
```

This installs `scikit-learn` (for Random Forest/OneClassSVM), `m2cgen` (for converting ML models to native Java code), and `pandas/numpy` for data wrangling. Keep your `requirements.txt` current for any future ML additions.

Step 3: Synthesize ML Models into Java (M2CGen)

A core feature of Aegis-IoT is zero-latency inference directly inside AnyLogic. We train our machine learning models in Python and use the `m2cgen` library to transpile them straight into pure Java bytecode.

Navigate to the `model` directory and execute the training pipelines:

```
Terminal / Command Prompt (ML Transpilation)
```

```
cd model python train_latency_random_forest.py python test_m2cgen4.py cd ..
```

What exactly did this do?

- `train_latency_random_forest.py` trained a Voting Regressor ensemble to predict packet latency based on packet sizes, inter-arrival times, and gateway queuing. It exported this directly into `OfflineAiPredictor.java`.
- `test_m2cgen4.py` trained a OneClassSVM model on normal traffic patterns to act as an Anomaly Radar (detecting DDoS bursts). It appended this logic to the same Java file.

- These intelligent models are now hardcoded natively into the Java source files, meaning AnyLogic does not need an API to perform AI inference!

Step 4: Inject UI and RL Features into the AnyLogic Core

To avoid endless manual GUI work in the AnyLogic editor, Aegis-IoT uses Python scripts to perform surgical XML injection into the AnyLogic project files.

Run the following commands from the root directory:

```
Terminal / Command Prompt (XML Mutation Engine)
```

```
cd injection_scripts python inject_multi_dashboard.py python  
inject_rl_and_gan.py cd ..
```

What exactly did this do?

- `inject_multi_dashboard.py`: Modified the AnyLogic startup sequence to automatically spawn four independent **Java Swing windows** when the simulation starts (`LatencyDash`, `SecurityDash`, `TelemetryDash`, and `EnergyDash`). This proves visualization can be handled flawlessly inside the AnyLogic JVM.
- `inject_rl_and_gan.py`: Hooked the Tabular Q-Learning agent (`QTableBalancer.java`) into the Gateway objects, allowing it to modify flow durations natively during simulation runtime.

Step 5: Launch the Core AnyLogic Simulation

It is time to bring the system online and prove self-contained execution.

1. Open the **AnyLogic** desktop application.
2. Go to **File > Open** and navigate to `Anylogic Models/FinalCCNProject1`.
3. Select `FinalCCNProject1.alpx` (or `.alp`).

4. Once loaded, look at the Project explorer panel on the left. Find the **Main** experiment.
5. Right-click the Main experiment and select **Run** (or click the green Play button on the toolbar).

Expected Native Behavior: A Simulation window will open. As configured by our injection scripts, **four custom Java Swing dashboards** will also pop up on your screen. The **SensorDevice** agents will begin generating MQTT packets, and you will see the **Gateway** agents actively routing them. The central database (**RUNTIME_LOGS**) will instantly start populating with native intelligence logs.

Step 6: Run the Federated Learning Scenario

Aegis-IoT includes a decentralized intelligence suite demonstrating FedAvg (Federated Averaging). This runs independently alongside the simulation to prove privacy-preserving edge training without relying on web services.

You will need to run the cloud aggregator and the edge nodes concurrently.

1. **Start the Cloud Server** (Open a new terminal):

```
Terminal / Command Prompt (Federated Server)
```

```
cd federated python cloud_server.py
```

This server awaits connections from multiple edge nodes to combine their trained weights locally.

2. **Start an Edge Node** (Open another new terminal):

```
Terminal / Command Prompt (Edge Node)
```

```
cd federated python edge_node.py
```

The edge node trains an SGDRegressor strictly on local (simulated) data partitions. It transmits ONLY the weights mathematically back to the cloud server.

3. **View the Results:** Once the communication rounds complete, visualize the training convergence:

```
Terminal / Command Prompt (Plot Output)
```

```
python plot_federated.py
```

This outputs Matplotlib graphs demonstrating how the decentralized models converged efficiently.

Final Validation

If you have followed all steps, your environment should now represent a fully functioning Aegis-IoT ecosystem completely centered around AnyLogic:

1. The AnyLogic simulator is visibly processing intelligent decision boundaries.
2. The transpiled M2CGen Java files are driving zero-latency logic with no backend web APIs.
3. The Java Swing GUI displays active radar, telemetry, and Q-Learning output directly from the JVM.
4. Complete logs are captured securely in local SQL databases.